

TPL.Dataflow

Харламов Михаил, 24Б44

The Task Parallel Library (TPL)

- Это набор общедоступных типов и API в пространствах имен `System.Threading` и `System.Threading.Tasks`
- Цель TPL — сделать разработчиков более продуктивными, упрощая процесс добавления параллелизма и асинхронности в приложения

Dataflow

- TPL Dataflow (библиотека `System.Threading.Tasks.Dataflow`) — это часть Task Parallel Library (.NET), предназначенная для создания высокопроизводительных асинхронных конвейеров обработки данных через модель потоков сообщений (message passing). Она упрощает параллельное выполнение CPU- и I/O-интенсивных задач, обеспечивая буферизацию, координацию и управление зависимостями между операциями без явных блокировок

Основное назначение

- Библиотека применяется для построения пайплайнов (линейных цепочек) или сетей блоков данных, где каждый блок обрабатывает сообщения асинхронно и передает их дальше. Идеально подходит для сценариев вроде обработки изображений, загрузки файлов, ETL-процессов или producer-consumer паттернов с высокой пропускной способностью и низкой задержкой. Она интегрируется с `async/await`, управляет буферами и позволяет настраивать параллелизм, избегая проблем с потоками.

Типы блоков

- Буферизующие (Buffering): `BufferBlock` (очередь FIFO), `BroadcastBlock` (последнее значение всем), `WriteOnceBlock` (одно значение).
- Исполнительные (Execution): `ActionBlock` (выполняет Action), `TransformBlock<TIn,TOut>` (Func с преобразованием), `TransformManyBlock<TIn,TOut>` (множественные выходы).
- Группирующие (Grouping): `BatchBlock` (батчи), `JoinBlock<T1,T2>` (туплы из источников), `BatchedJoinBlock` (батчи туплов)

Пример



```
var buffer = new BufferBlock<byte[]>();  
  
var compressor = new TransformBlock<byte[], byte[]>(bytes => Compress(bytes));  
  
var writer = new ActionBlock<byte[]>(bytes => outputStream.Write(bytes, 0, bytes.Length));
```

LinkTo



```
this.buffer.LinkTo(this.compressor);  
this.buffer.Completion.ContinueWith(_ => this.compressor.Complete());  
this.compressor.LinkTo(this.writer);  
this.compressor.Completion.ContinueWith(_ => this.writer.Complete());
```

С распараллеливанием



```
var buffer = new BufferBlock<byte[]>(new DataflowBlockOptions { BoundedCapacity = 100 });
var compressorOptions = new ExecutionDataflowBlockOptions
{
    MaxDegreeOfParallelism = 4,
    BoundedCapacity = 100
};
var compressor = new TransformBlock<byte[], byte[]>(bytes => Compress(bytes), compressorOptions);
var writerOptions = new ExecutionDataflowBlockOptions
{
    BoundedCapacity = 100,
    SingleProducerConstrained = true
};
var writer = new ActionBlock<byte[]>(bytes => outputStream.Write(bytes, 0, bytes.Length),
writerOptions);
```


ИСТОЧНИКИ

- <https://learn.microsoft.com/ru-ru/dotnet/standard/parallel-programming/task-parallel-library-tpl>
- <https://learn.microsoft.com/ru-ru/dotnet/standard/parallel-programming/dataflow-task-parallel-library>
- <https://habr.com/ru/articles/138531/>
- Код: <https://github.com/mikhail-kharlamov/spbu-3-semester-programming/tree/MyNUnit/tpl-dataflow>